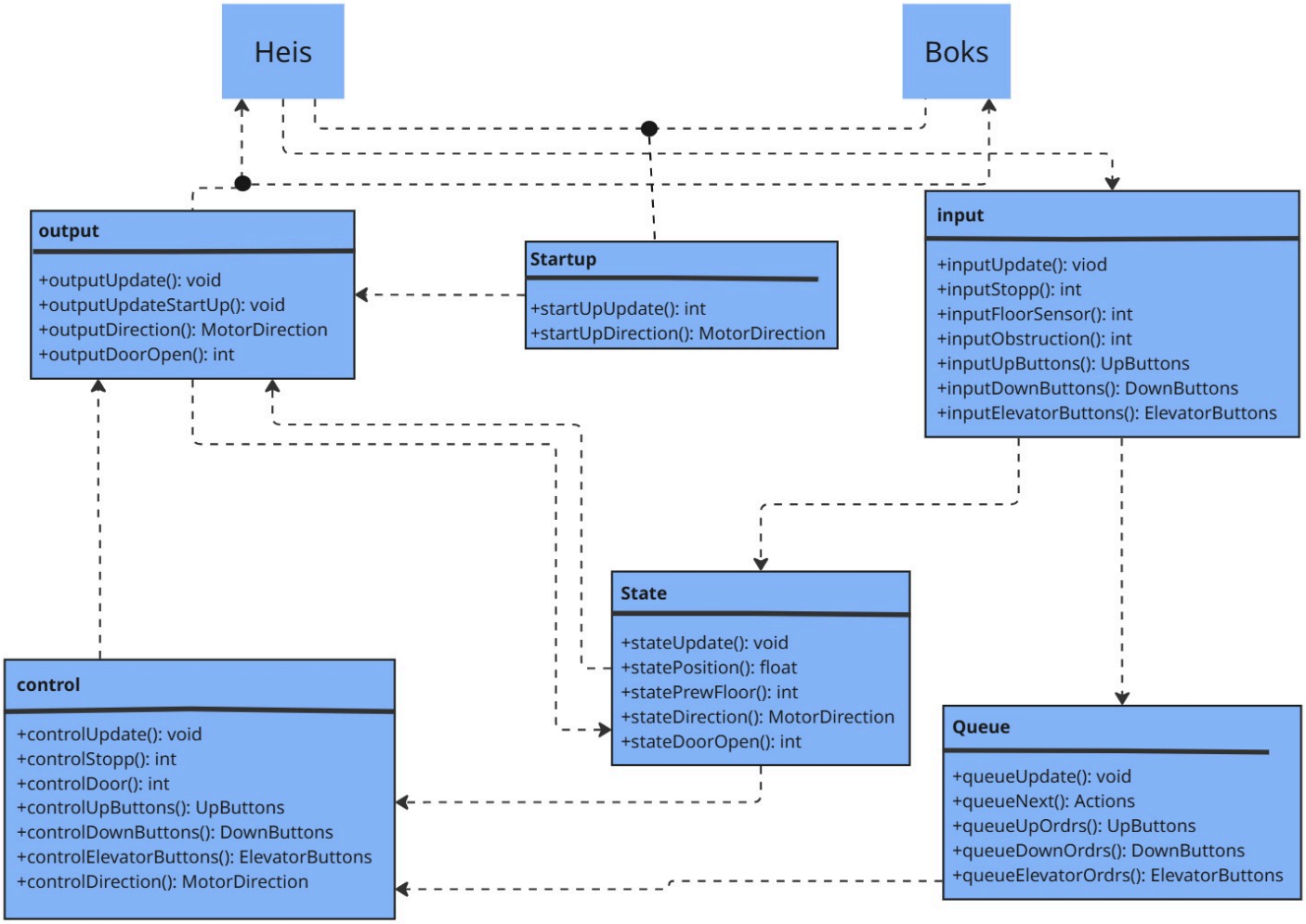
Heislab TTK4235 Gruppe 55

Emma Øverbø Johannessen, Jørand Runningen

16. mars 2026



Figur 1: Klassediagram som beskriver systemet vi har implementert.

UML og V-modellen

Vi har lagd et klassediagram over modulene vi har delt systemet i, to sekvensdiagramer som viser hvordan modulene innteragerer under henholdsvis oppstart og vanlig kjøring, og et tilstandsdiagram som viser styringen av motoren og døren.

Klassediagram

Figur 1 viser klassediagrammet vårt. Systemet består av seks moduler, som er implementert som hver sin c-fil i prosjektet. output-modulen benyttes til både oppstart og vanglig kjøring av heisen, mens startUp-modulen kun brukes i oppstart. De restrenende modulene brukes kun under vanlig kjøring. De ulike modulene har ulike ansvarsområder.

StartUp-modulen

StartUp-modulen henter informasjon fra heisen om hva etasjesensorene viser og bestemmer retningen heisen skal kjøre.

Input-modulen

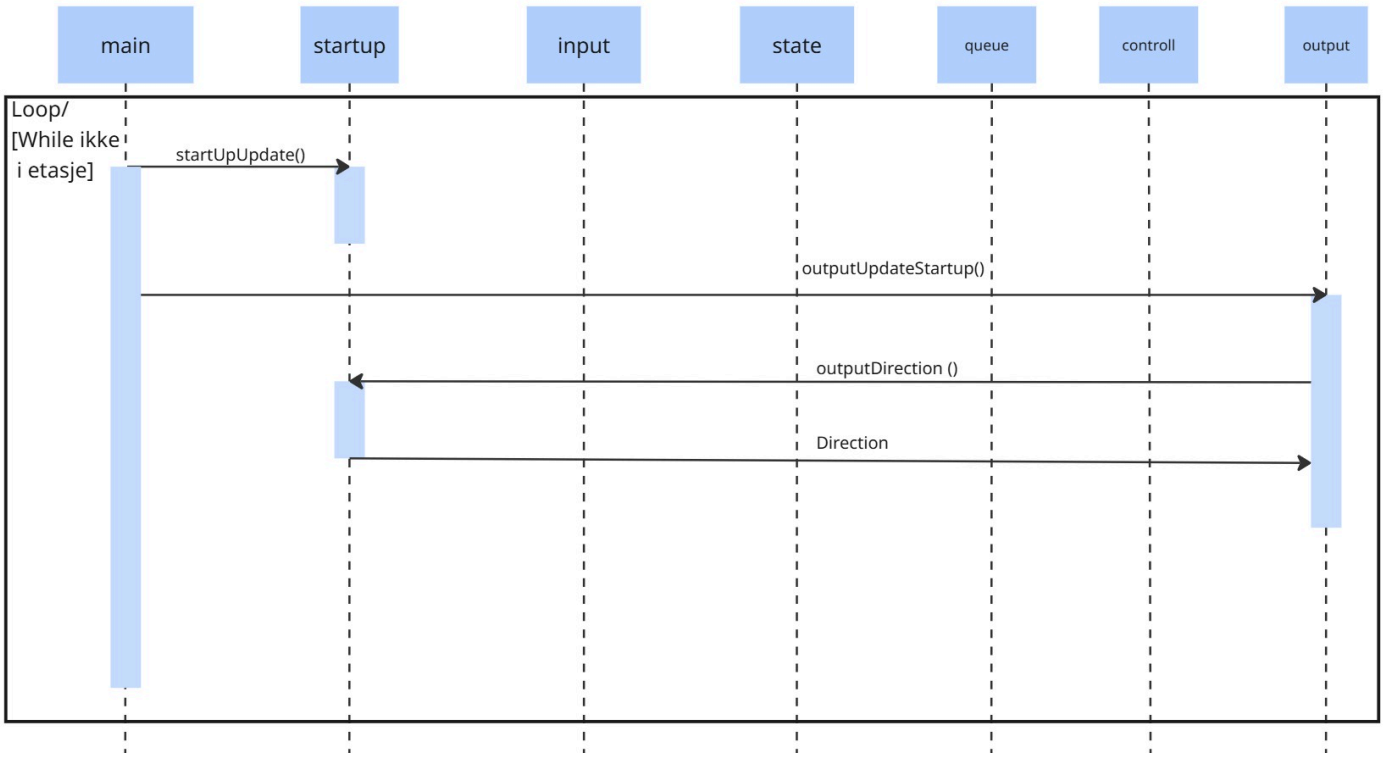
Input-modulen henter informasjon fra heisen og kontrollboksen, altså hvilke knapper som er trykket inn og hva etasjesensorene viser.

State-modulen

State-modulen har i oppgave å vite posisjonen til heisen, hvilken vei motoren beveger seg, og om døren er åpen eller lukket.

Queue-modulen

Queue-modulen beregner hva den neste handlingen til heisen skal være, altså som den skal stå stille, stå stille med åpen dør eller eventuelt hvilken etasje den skal bevege seg til.



Figur 2: Sekvensdiagram som viser interaksjonene mellom modulene under oppstart av heisen.

Control-modulen

Control-modulen beregner hvilke lys som skal være på og hvilken retning motoren skal bevege seg.

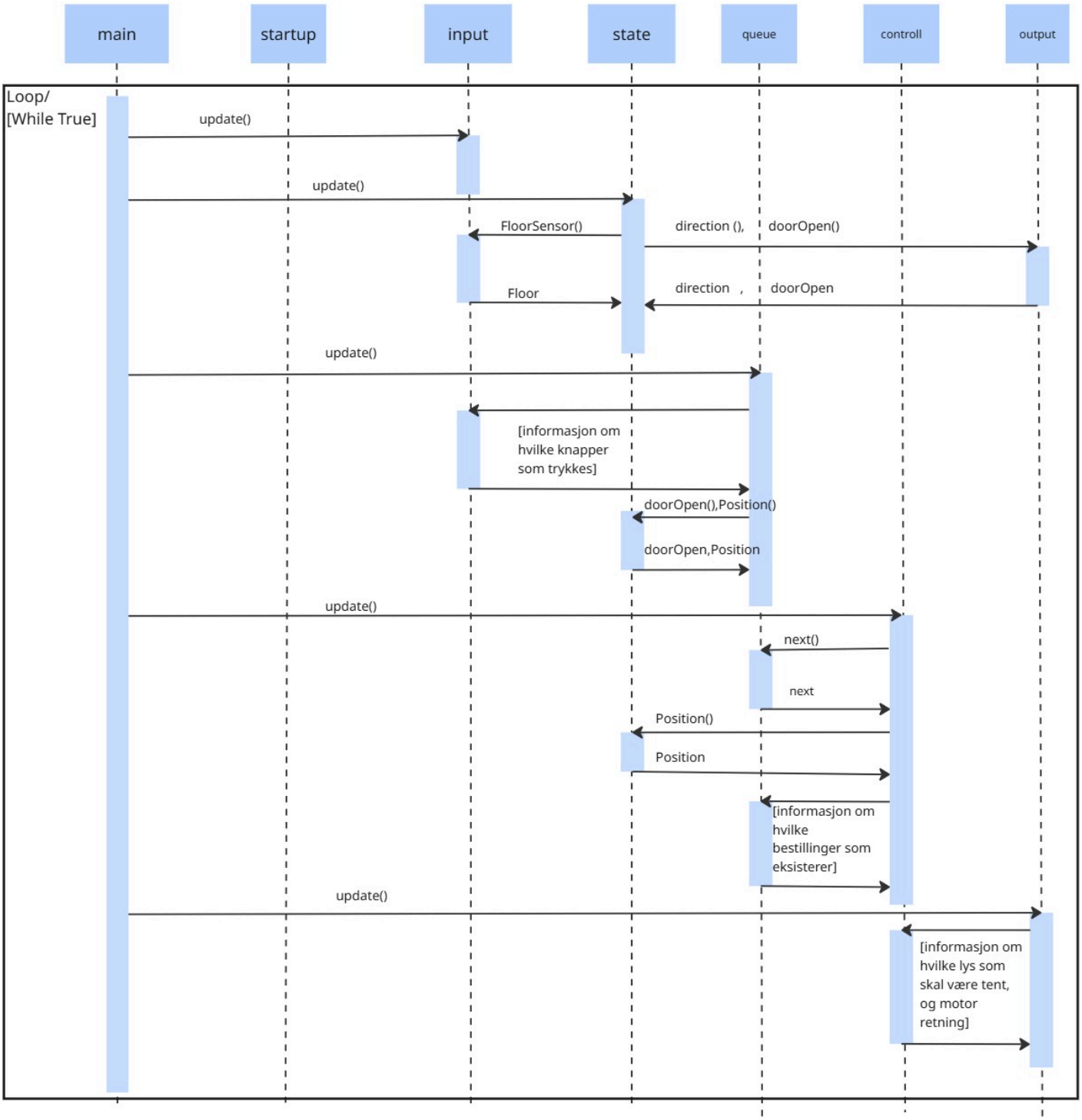
Output-modulen

Output-modulen videresender til heisen og kontrollboksen hvilke lys som skal være på og hvilken vei motoren skal bevege seg.

Sekvensdiagramer

Sekvensdiagram for oppstart

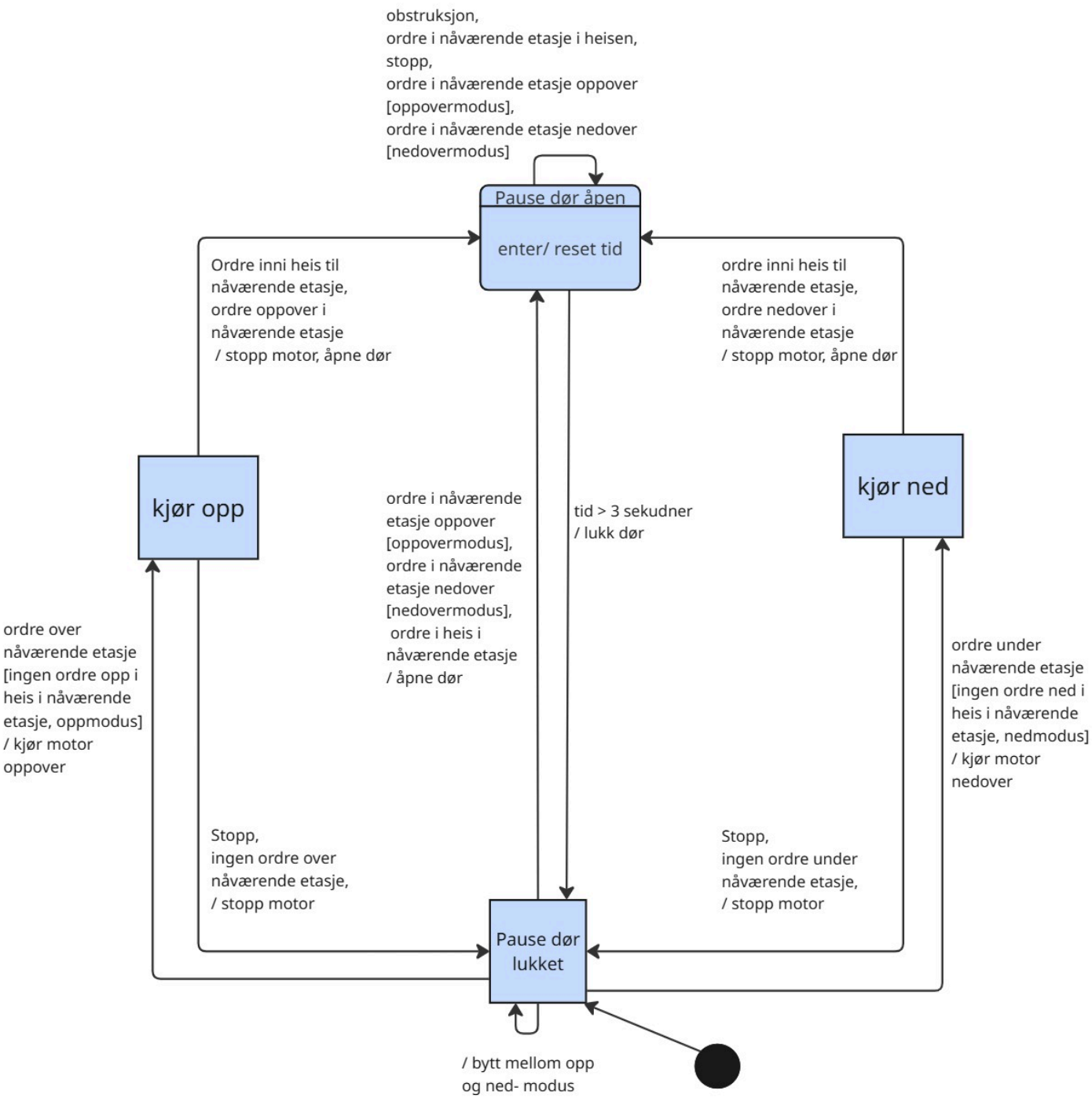
Figur 2 viser sekvensdiagrammet for oppstart. Oppstartfasen kjører til heisen kommer i en kjent tilstand, altså befinner seg i en etasje. Under oppstart får main startup- og output-modulen til å oppdatere sin interne tilstand. At de oppdaterer sin interne tilstand betyr at de løser oppgavene sine, som beskrevet i delen om klassediagrammet. Se klassediagram. StartUp returnerer nedover som motorretning til output, helt til heisen når en etasje, da skal motoren stoppe.



Figur 3: Sekvensdiagram som viser interaksjonen mellom modulene under vanlig kjøring av heisen. Merk at funksjonsnavnene er forkortet. Alle funksjonene har en prefiks som er navnet på modulen den tilhører. F.eks skal første kall fra main være inputUpdate() ikke update().

Sekvensdiagram for vanlig kjøring

Figur 3 viser hvordan modulene intragerer under vanlig kjøring. main får de ulike modulene til å oppdatere sin interne tilstand. Når modulene skal oppdatere sin interne tilstand, henter de informasjon fra de modulene de trenger informasjon fra. At en modul oppdaterer sin interne tilstand betyr at de løser de oppgavene de har ansvar for, som beskrevet i delen om klassediagrammet. Se klassediagram.



Figur 4: Tilstandsdiagram som beskriver hvordan systemet vi har implementert styrer motoren og døren.

Tilstandsdiagram

Figur 4 viser hvordan vi styrer motoren og døren. Heisen har fire tilstander, stå stille med lukket dør, stå stille med åpen dør, kjør oppover og kjør nedover. Heisen har dessuten to moduser: opp- og ned-modus. Dersom heisen er i opp-modus betjener den bare ordre til personer som befinner seg i samme etasje som heisen eller i en etasje over heisen. Den betjener kun ordrene til folk som vil oppover, men dersom ingen vil oppover betjener heisen den øvreste ordren som vil nedover. Dersom det ikke finnes en eneste ordre over posisjonen til heisen, bytter heisen til ned-modus. Ned-modus virker som opp-modus, men motsatt.

Mens heisen venter på å betjene ordre står den stille og oscillerer den mellom opp- og ned-modus.

Refleksjoner

Refleksjoner rundt UML og V-modellen

Startfasen av implementasjonen

Vi begynte med å skissere hvilke moduler vi ville dele systemet opp i. Ideen vår var å lage moduler med enkle sammhandlinger og konkrete oppgaver, slik at vi kunne feilsøke modulene helt uavhening av hverandre, ved å kontrollere input til hver modul og se på output-et. Dette skulle gjøre enhetstesting eneklt, og ettersom modulene har enkle sammhandlinger håpet vi integrasjonstestingen ville bli enkel. For å ha en ide om hva hver enkelt modul skulle gjøre, fordelte vi de ulike punktene i kravspesifiaksjonen på de ulike modulene. Deretter skisserte vi også noen sekvensdiagrammer for å finne ut hvordan modulene kunne virke sammen. På dette stadiet revurderte vi noen av modulene våre og hvordan vi hadde fordelt ansvaret på de ulike modulene.

Noen fornuftige nøkkelideer for systemet vi kom frem til i dette steget var: 1) main forteller hver modul når den skal oppdatere sin interne tilstand, for øverig endres ikke tilstanden i en modul. På denne måten er den interne tilstanden i modulen forutsigbar. 2) modulene oppdaterer intern tilstand og returnerer deretter til main, ingen moduler har dermed lov til å oppholde progammet mer enn et kort øyeblikk. Dette sikrer at hver modul får oppdatert sin interne informasjon ofte, slik at man unngår alle problemer med at man f.eks ikke rekker å registrere at en knapp trykkes, eller at heisen passerer forbi en etasje.

Midtfasen av implementasjonen

Etter å ha bestemt oss for hvilke moduler vi skulle ha, bestemte vi oss for nøyaktig hvilken informasjon de skulle utveksle, og formatet på informasjonen, slik at vi kunne programere modulene hver for seg. Dette steget gjorde vi i stor grad uten å tenke på hvilken funksjonalitet c tilbyr, og vi kikket heller ikke på hvordan kommunikasjon med heisen var implementert. Dette var en tabbe som vi i etterkant måtte rette opp i, ved for eksmepel å definere noen felles struct-er og enum-er som hele prosjektet forholder seg til. Noen ting har derimot blitt med gjennom hele prosjektet, for eksempel bruker systemt vårt 1-4 som etasjer, i motsetning til den utdelte koden, som benytter 0-3.

Vi oppdaget mens vi programerte modulene at én av modulene var redundant, så vi fjernet den, og ente opp med modulene beskrevet tildligere i rapporten.

Sluttfasen av implementasjonen

Når vi hadde skrevet all koden, fant vi ut at det var betydelig jobb å skrive kode for teste hver modul hver for seg. Vi droppet derfor enhetstesting og gikk rett på implementasjonstestning. Etter å ha rettet opp i noen syntaksfeil kompilatoren sa fra om, var koden nesten helt problemfri. Siden hver modul har realtivt tydelige ansvar, var det enkelt å spore feil til korrekt modul. Og siden kommunikajsonen mellom modulene er enkel, er systemet vårt lite komplekst, vi hadde derfor ikke problemer med feil som oppstår i obskure spesialtilfeller. Etter litt enkel

implementasjonstestning hadde vi et fungerende system.

Vi oppdaget derimot at vi hadde tenkt litt feil på hvilken rekkefølge man burde prioritere etasjene, men siden queue-modulen har ansvaret for å bestemme hva heisen skal gjøre, og resten av systemet virker uavhening av queue-modulen, kunne vi endre queue-modulen til korrekt oppførsel uten at dette ga bug-er i resten av systemet.

Vurdering av vår bruk av UML og V-modellen

Vi hadde stor nytte av å dele prosjektet i mindre moduler med tydelige oppgaver, slik V-modellen legger opp til. Modulene vi endte opp med var i veldig stor grad enkle å oversette til kode, med unntak av queue-modulen, og det er relativt tydelig hva de ulike modulene skal gjøre. For å klare å dele prosjektet på denne måten hadde vi stor nytte av klassediagrammet vi satte opp, og noen sekvensdiagrammer vi skisserte. Systemet virket derimot så godt at det var fornuftig å ignorere enhetstesting, vi har altså til en viss grad tilpasset V-modellen litt.

Vi benyttet oss derimot ikke av tilstandsdiagramet beskrevet tidligere i rapporten. Det er lagd i etterkant, etter at heisen fungerte. Da vi planla så vi ikke hva vi skulle bruke det til, vi tror derimot det ville vært veldig nyttig for å få et enda enklere system. Koden til queue-modulen gjør i praksis det tilstandsdiagramet sier, men den er ikke tydelig implementert som en tilstandmaskin med klart definerte tilstander. Dessuten er noe av logikken fordelt til control også, så det er mye vanskligere å ha god oversikt over hvordan queue- og control-modulen sammhandler om å realisere oppførselen til en tilstandsmaskin.

Så erfaringen vår er at UML-skjemaene var svært nyttige ettersom de positive sidene av prosjektet i stor grad har opphav i fornuftig bruk av UML og V-modellen. Og hadde vi innsett nytten av å sette opp en tilstandsdiagram før vi skrev koden til queue- og control-modulen, ville prosjektet blitt lettere å gjennomføre, og mer oversiktelig i etterkant.

Vår bruk av KI

Vi har skrevet koden vår selv, ingen ting er kopiert fra KI. Vi har derimot brukt KI til å tolke feilmeldinger og søke opp syntaks. Vi har altså brukt den for raskere søk enn det er å finne tilsvarende informasjon på stackoverflow og lignende. Så KI har i praksis gjort det litt raskere å skrive kode, ved at søk har tatt kortere tid en hva tradisjonell googeling ville gjort. Bortsett fra det, har ikke KI hatt noen stor effekt på prosjektet.

Refleksjoner rundt robusthet, skalerbarhet og vedlikeholdbarhet

Siden vi brukte V-modellen og delte prosjektet opp i mindre moduler, som interagerer på lite komplekse måter, er det enkelt å legge til ny funksjonalitet eller endre gamel funksjonalitet. Man må bare finne ut hvilken modul endringen hører inn under, og så lenge brukergrensesnittet til modulen er uforandret vil prosjektet fungere. Vi har dessuten forøskt å ha en systematisk konvensjon på navngivning av variabler, med beskrivende navn, slik at det i etterkant skal være enkelt å sette seg inn i hva koden gjør.

Det er derimot noen problemer med koden med hensyn på vedlikeholdbarhet. Det viktigste punktet er at vi ikke har skrevet noen dokumentasjon. For øyeblikket er det relativt tydelig for oss hva de ulike modulene gjør, men det er ikke skrevet ned, så andre personer, eller en senere versjon av oss selv, vil ha større problemer med å sette seg inn i koden, enn om vi hadde skrevet dokumentasjon.

Det andre problemet er at vi ikke bare gjorde fornuftige valg da vi valgte hva de ulike modulene skulle ha ansvar for. Dette har resultert i en del redundant kode, spesielt i output-modulen. Den sørger for at døren ikke er åpen i fart, at heisen ikke kjører utenfor etasje 1-4 osv. Dette sørger derimot allerede queue- og control-modulen for. Output-modulen har altså veldig mange linjer med unødvendig kode, som

gjør koden vanskligere å forstå enn nødvendig. Dette fører dessuten til at det er flere ulike steder i koden man kan ha problemer med åpning av dør og lignende. Dette bryter med prinssippet vårt om at modulene skal ha tydelig fordelte ansvar, slik at feil enkelt kan spores til riktig modul.

Et annet problem med output-modulen er at den er skrevet for å kommunisere minst mulig med heisen, den vil for eksmepel aldri forsøke å slå på lys som allerede er på. Hensikten var å unngå at programflyten skulle stoppe unødvendig lenge i output-modulen, slik at systemet for eksmpel ikke rakk å registrere at heisen paserte forbi en etaje. Etter å ha snakket med andre grupper virker det derimot ikke som dette er en bug de har problemer med. Systmet vårt med å sende færrest mulig forespørsler til heisen bidrar altså til å gjør output-modulen mer komplisert, uten å løse et reelt problem.

Et siste problem med hensyn på vedlikeholdbarhet er som kommentert tidligere at queue-modulen ikke er implementert som en tilstandsmaskin, dette gjør den mer uoversiktelig enn den trenger å være.

Refleksjon rundt bruk av git

Vi har brukt git gjennom hele projektet for å muliggjøre arbeid hver for oss. Dette har fungert bra da modulene er selvstedinge og vi kunne jobbe med en modul uten å påvirke andre deler ved koden. Videre har det også vært nyttig at arbeidet ikke bare blir lagret på sanntidspc-en. På denne måten unngikk vi potensielle problemer med at arbeidet ble slettet.

Vi brukte git til å dele opp i ulike branches. Dette var bra for å holde oversikt over endirgnene som ble gjort, men det føltes også litt begrendense når vi skulle gjøre endringer på tvers av de ulike branchene.

Siden det var første gang vi brukte git ble det oppfattet som litt knotete å jobbe med, men vi ser absolutt nytten av å bruke git og syntaksen gikk seg til etterhvert.